

Architecture Diagrams and Decision Records Reviewers Trust

How to draw cloud architecture diagrams that answer questions before they're asked, and a lightweight architecture decision record (ADR) template for the choices you'll be asked to defend.

September 26, 2026 · 4 min read

<https://brianpfeil.com/architecture/diagrams-and-decision-records/>

Two artefacts do most of the work in an architecture review: the **diagram**, which shows what you're building, and the **decision record**, which shows why. Get both right and the meeting is about trade-offs, not archaeology.

Diagrams

Use the reference-architecture style

Draw in the style of your cloud provider's reference architectures, with the provider's official icons. Reviewers read that visual language fluently, and it stops arguments about what a box means. Free tools are fine — draw.io (including its VS Code extension) ships the major providers' icon sets.

Number the flow

Put a number on every arrow of the main path — 1, 2, 3 — and explain each step in a short list beside the diagram. A reviewer can then say "at step 4, what happens if..." and everyone is looking at the same place.

Draw the boundaries

Most security and networking questions are really questions about boundaries. Show them:

Boundary	Why it matters
Internet · your company · on-premises	What's exposed, and what crosses the edge

Boundary	Why it matters
Other clouds · partners · SaaS	Who you depend on and how you connect
Accounts · regions	Blast radius, data residency, DR
Networks · availability zones · subnets (public/private)	Reachability and high availability
Devices · users	Who actually starts each flow

One diagram can't do everything

I ask for a small set rather than one poster:

1. **Core solution** — the numbered request and data flow.
2. **Build and deploy** — commit to production.
3. **Availability, backup and recovery** — what fails over to where, and what gets restored from what.
4. **Vendor or partner lifecycle** — how their updates reach you, if relevant.

If a cross-cutting concern — secrets, alarms, backups, cost — isn't on any diagram or page, reviewers will assume it hasn't been designed. Usually they're right.

Decision records

An **architecture decision record (ADR)** is a short document that captures one significant decision: what you chose, what you didn't, and why. They're cheap to write, and they save hours when someone asks "why on earth did we...?" a year later — often you.

Write one whenever you choose between real options: a database, a compute rung, a vendor, an integration style, or a concession you plan to fix later.

A template

```
# ADR-NNN: <short decision title>
```

- Status: proposed | accepted | superseded by ADR-`MMM`
- Deciders: <roles, not just names>
- Date: `YYYY-MM-DD`

Context and problem

What forces are at play? What question are we answering?

Decision drivers

- e.g. time to deliver, team skills, cost, risk, compliance

Options considered

1. Option A
2. Option B
3. Option C

Decision

We chose <option>, because <the drivers it satisfies best>.

Positive consequences

Negative consequences (and how we'll live with them)

Pros and cons of each option

Option A

- Good, because ...
- Bad, because ...

Tips that make ADRs useful

- **Include the options you rejected.** The rejected options are what stop the same debate happening again.
- **Name the drivers before the decision.** It keeps the choice honest.
- **Supersede, don't edit.** When a decision changes, write a new record and link the old one. The history is the value.
- **Record concessions with a date.** "We'll run it on servers for now" is a decision; "...and move to serverless containers by Q3" makes it a plan.

Recommendations in two tiers

When I write up a review, recommendations usually come as:

- **Primary** — the target architecture we'd want if nothing held us back.
- **Interim** — a step the team can ship now, often "run it as-is on managed compute", that moves toward the primary without blocking delivery.

Where there are several candidate services, I list them **in the order to evaluate**, most managed and cheapest first — so the team stops at the first one that works rather than the first one they know.

The target-state document

Once the team has chosen, they write a **target-state architecture document** — the agreed architecture, diagrams and decisions in one place — and get it approved before the build starts. They come back only when something significant changes. Store it, versioned, next to everything else in one predictable place linked to the work item.

Related: [How to prepare for an architecture review.](#)